

AI Intern Take-Home Assignment

Multi-Step Research Assistant API

Format: private GitHub repo + live deployed link

1. Problem Statement

Build a backend service that can answer questions over a set of documents by reasoning about them, not just by pattern-matching a single retrieval pass to a single generation call. The service should be able to handle questions that a naive “embed the question, fetch top-k chunks, ask the LLM” pipeline would get wrong or answer badly.

Pick any document set you like for the knowledge base, in any format: PDF, Word, HTML, plain text, scanned images, whatever's convenient. It just needs to be substantial and varied enough that shallow retrieval genuinely struggles with some of the questions you could reasonably ask of it.

If a question can't be answered from the knowledge base, the system should be able to fall back to searching the internet for an answer rather than simply saying it doesn't know.

The service is exposed as an API, optionally with a simple interface to interact with it. A polished or good-looking UI is not important for this assignment; we care about what the system can actually do, not how it looks.

2. Expected Results

- An endpoint to add documents to the knowledge base.
- An endpoint to ask a question and get back an answer, along with an indication of what in the knowledge base the answer is based on.
- The system should hold up under questions that are compound, ambiguous, under-specified, or that build on earlier turns in the same conversation.
- The system should not confidently answer something the knowledge base doesn't actually support — in that case, it should look further before giving up.
- Documents in different formats should all work through the same ingestion path without needing separate handling from the user's side.
- The reasoning behind an answer should be a genuine multi-step process reflected in your implementation, not a single prompt doing all the work.

3. Required Tech Stack

- FastAPI
- LangChain
- LangGraph
- A vector database (any of your choice)
- The application must be deployed and reachable at a public URL.

4. Submission Checklist

Please submit all of the following, incomplete submissions may be evaluated as-is without follow-up:

- Link to a public (or invite-shared) GitHub repository.
- Link to the live deployed application, with the API docs reachable.

- A short README covering: setup/run instructions, what documents you used for the knowledge base, and any trade-offs you made given the time limit.

Questions during the assignment? Reach out any time, asking a good clarifying question is a positive signal, not a negative one.